

MINISTRY OF EDUCATION AND RESEARCH
OF THE REPUBLIC OF MOLDOVA

Technical University of Moldova
Faculty of Computers, Informatics and Microelectronics
Department of Software and Automation Engineering

Report

Laboratory work No. 11

The Race and the Semaphore

Coșneanu Corina, st. gr. FAF-241

Checked by:
Reițman Patricia, *University Assistant*
Department of Software and Automation Engineering,
FCIM Faculty, UTM

Chișinău – 2026

Contents

1 LABORATORY OBJECTIVES AND THEORY **1**
1.1 Objective 1
1.2 Theoretical Background 1

2 PRACTICAL WORK AND OBSERVATIONS **2**
2.1 Part 1: See the race 2
2.1.1 Observe Answers (Part 1) 2
2.2 Part 2: Fix it: mutex, then semaphore 4
2.2.1 Observe Answers (Part 2) 6
2.3 Part 3: A semaphore as a signal 7
2.3.1 Observe Answers (Part 3) 8
2.4 Part 4: A semaphore as a counter: the parking lot 9
2.4.1 Observe Answers (Part 4) 10

3 CONCLUSIONS **12**

1 LABORATORY OBJECTIVES AND THEORY

1.1 Objective

The objective of this laboratory work is to see a race condition between two threads that change the same variable, to fix it with a mutex and with a semaphore, and to use the same semaphore in three different ways: as a lock, as a signal and as a counter.

1.2 Theoretical Background

The threads of a process share the same memory, so they can read and write the same global variables. This is useful, but it is also the source of the problems studied in this work:

- **Race condition.** A race appears when two threads use the same variable at the same time, at least one of them writes it, and the result depends on the order in which their steps are interleaved. An operation that looks like one step in C, such as `counter++`, is in fact several steps for the processor, and another thread can run between them.
- **Critical section.** The part of the code that uses the shared variable is called a critical section. The program is correct only if one thread at a time can be inside it (mutual exclusion).
- **Mutex.** A mutex is a lock with two operations: `pthread_mutex_lock` lets one thread pass and makes the others wait, and `pthread_mutex_unlock` lets the next one in.
- **Semaphore.** A semaphore is a number that never goes below 0. The operation `sem_wait` takes one unit, and waits if the value is 0; the operation `sem_post` gives one unit back and wakes up a waiting thread. The start value decides how it is used: started at 1 it is a lock, started at 0 it is a signal (a closed door that another thread opens), and started at N it is a counter of N free resources.

All the programs were compiled with `gcc -Wall -pthread` and without `-O2`, because the optimizer could hide the race of Part 1.

2 PRACTICAL WORK AND OBSERVATIONS

2.1 Part 1: See the race

In the program `race.c` two threads each add 1 to the same counter, one million times, with no protection. The right answer is 2 000 000.

```c
#include <stdio.h>
#include <pthread.h>

#define N 1000000

long counter = 0;                   /* shared by both threads */

void *add(void *arg) {
    for (int i = 0; i < N; i++)
        counter++;                  /* read, add 1, write back: three
            steps */
    return NULL;
}

int main(void) {
    pthread_t a, b;
    pthread_create(&a, NULL, add, NULL);
    pthread_create(&b, NULL, add, NULL);
    pthread_join(a, NULL);
    pthread_join(b, NULL);
    printf("counter = %ld (expected %d)\n", counter, 2 * N);
    return 0;
}
```

The program was run three times. Figure 2.1 shows that the result is wrong and different at every run: it is always smaller than the expected 2 000 000, which means that many additions were lost.

```
counter = 1035220 (expected 2000000)
counter = 1160290 (expected 2000000)
counter = 1980459 (expected 2000000)
```

Figure 2.1: Three runs of `race`: the counter is wrong and different every time

2.1.1 Observe Answers (Part 1)

1. **The three steps of `counter++`:** (1) *load*: read the value of `counter` from memory into a register; (2) *add*: add 1 to the register; (3) *store*: write the register back into `counter` in memory.

2. **How one addition is lost:** both threads can do the load step before either of them does the store step. For example, with `counter = 5`: thread A loads 5, thread B loads 5, A adds 1 and stores 6, B adds 1 and also stores 6. Two additions were made, but the counter grew only by 1.

3. **Why the result is different at every run:** the result depends on the exact moments when the two threads are interleaved, and the scheduler decides this differently at every run (it depends on timing, on the processor cores and on the other processes of the system). The number of lost additions is therefore different every time.

2.2 Part 2: Fix it: mutex, then semaphore

2.1 A mutex around `counter++`. In `mutex.c` the addition is placed between a lock and an unlock operation. The line between them is the critical section: only one thread may be inside. The rest of the program is the same as `race.c`.

```
long counter = 0;
pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;

void *add(void *arg) {
    for (int i = 0; i < N; i++) {
        pthread_mutex_lock(&lock);      /* only one thread past
           this line */
        counter++;                      /* the critical section */
        pthread_mutex_unlock(&lock);    /* let the other one in */
    }
    return NULL;
}
```

2.2 The same lock, made from a semaphore. In `sem.c` the mutex is replaced by a semaphore that starts at 1. The first thread that calls `sem_wait` changes the value from 1 to 0 and goes in; the other one finds 0 and waits until `sem_post` puts the value back to 1.

```
long counter = 0;
sem_t s;

void *add(void *arg) {
    for (int i = 0; i < N; i++) {
        sem_wait(&s);           /* 1 -> 0: I go in; at 0: wait */
        counter++;
        sem_post(&s);           /* 0 -> 1: the next one may go in
           */
    }
    return NULL;
}

int main(void) {
    sem_init(&s, 0, 1);         /* 0 = shared between threads, 1 =
       start value */
    ...
    sem_destroy(&s);
    return 0;
}
```

Both programs were run three times each. Figure 2.2 shows that the answer is now always 2 000 000.

```
counter = 2000000 (expected 2000000)
counter = 2000000 (expected 2000000)
counter = 2000000 (expected 2000000)
counter = 2000000 (expected 2000000)
counter = 2000000 (expected 2000000)
counter = 2000000 (expected 2000000)
```

Figure 2.2: Three runs of `mutex` and of `sem`: the counter is always 2 000 000

2.3 The price of the lock. The three programs were measured with the `time` command (Figure 2.3). The unprotected version is the fastest, but its answer is wrong. The two protected versions give the right answer, but they are clearly slower.

```
counter = 1717754 (expected 2000000)

real    0m0.018s
user    0m0.016s
sys     0m0.001s
counter = 2000000 (expected 2000000)

real    0m0.121s
user    0m0.036s
sys     0m0.026s
counter = 2000000 (expected 2000000)

real    0m0.201s
user    0m0.106s
sys     0m0.234s
```

Figure 2.3: Run time of `race`, `mutex` and `sem` measured with `time`

A semaphore that starts at 2. To answer the second question, a copy of the program (`sem2.c`) was made, in which the only change is the start value: `sem_init(&s, 0, 2)`. Figure 2.4 shows that the result is wrong again.

```
    sem_init(&s, 0, 2);         /* 0 = shared between threads, 1 = start value *
/
counter = 1772752 (expected 2000000)
counter = 1782849 (expected 2000000)
counter = 1831966 (expected 2000000)
vboxuser@lab1:~/os-2026-CosneanuCorina/lab11$
```

Figure 2.4: Three runs of `sem2` (start value 2): the race is back

2.2.1 Observe Answers (Part 2)

1. **Is the answer always 2 000 000?** Yes. In all three runs of `mutex` and in all three runs of `sem` the result was `counter = 2000000 (expected 2000000)` (Figure 2.2).

2. **What happens with `sem_init(&s, 0, 2)`?** The semaphore has two units, so both threads can pass `sem_wait` at the same time and both are inside the critical section together. The semaphore does not protect anything anymore and the race is back: the result is again smaller than 2 000 000 and different at every run (Figure 2.4).

3. **Why is the locked version slower?** Every single addition now also pays for a lock and an unlock operation (2 000 000 pairs in total), which cost much more than `counter++` itself. In addition, the two threads cannot work at the same time anymore: when the lock is taken, the other thread has to wait, and it may be put to sleep and woken up by the kernel.

2.3 Part 3: A semaphore as a signal

In this part the problem is not two threads at once, but the order: thread B must read the message only after thread A has written it. In `order.c` the semaphore `ready` starts at 0, so it is a closed door: B waits at `sem_wait` until A calls `sem_post`.

```c
char message[32];
sem_t ready;                    /* starts at 0: nothing is ready yet */

void *writer(void *arg) {
    sleep(1);                   /* pretend the work takes a while */
    strcpy(message, "hello");
    printf("A: message written\n");
    sem_post(&ready);           /* 0 -> 1: wake up B */
    return NULL;
}

void *reader(void *arg) {
    printf("B: waiting for the message...\n");
    sem_wait(&ready);           /* blocks while the value is 0 */
    printf("B: I read \"%s\"\n", message);
    return NULL;
}

int main(void) {
    sem_init(&ready, 0, 0);
    pthread_t a, b;
    pthread_create(&b, NULL, reader, NULL);   /* B starts first, on purpose */
    pthread_create(&a, NULL, writer, NULL);
    pthread_join(a, NULL);
    pthread_join(b, NULL);
    sem_destroy(&ready);
    return 0;
}
```

Figure 2.5 shows the output. B is started first and prints its first line at once, then nothing happens for one second, and only after A writes the message does B read `"hello"`. The order is correct even though B started first.

```
B: waiting for the message...
A: message written
B: I read "hello"
```

Figure 2.5: Output of `order`: B reads the message only after A has written it

For the first question, a copy of the program (`order1.c`) was made with the start value changed to 1: `sem_init(&ready, 0, 1)`. Its output is shown in Figure 2.6.

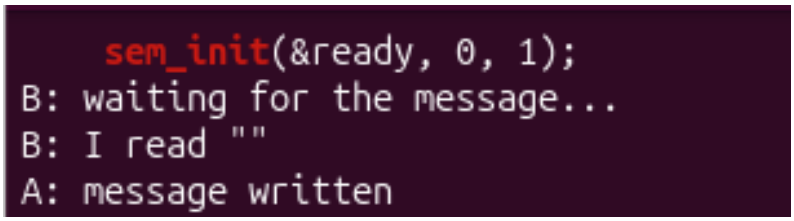


Figure 2.6: Output of `order1` (start value 1): B reads an empty message before A writes it

2.3.1 Observe Answers (Part 3)

1. **What does B read with the start value 1, and why?** B prints `B: I read ""`, an empty message, and it prints it at once, before `A: message written` (Figure 2.6). The value 1 means that the door is already open: `sem_wait` takes the unit and returns immediately, so B does not wait for A and reads `message` before A has written it, while the array is still empty.

2. **What does a semaphore that starts at 0 mean?** It means "nothing is ready yet": it is a closed door, and whoever calls `sem_wait` must wait until another thread calls `sem_post`.

2.4 Part 4: A semaphore as a counter: the parking lot

In `parking.c` there are six cars (threads) and three places. The semaphore `places` starts at 3 and counts the free places. The mutex is there only so that the variable `inside` and the `printf` stay together.

```c
#define PLACES 3
#define CARS   6

sem_t places;                                   /* free places */
pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;
int inside = 0;                                 /* only for
   printing */

void *car(void *arg) {
    long id = (long)arg;
    sem_wait(&places);                          /* take a place, or
        wait */
    pthread_mutex_lock(&lock);
    inside++;
    printf("car %ld enters  (inside: %d)\n", id, inside);
    pthread_mutex_unlock(&lock);

    sleep(1);                                   /* parked */

    pthread_mutex_lock(&lock);
    inside--;
    printf("car %ld leaves  (inside: %d)\n", id, inside);
    pthread_mutex_unlock(&lock);
    sem_post(&places);                          /* give the place
       back */
    return NULL;
}

int main(void) {
    sem_init(&places, 0, PLACES);
    pthread_t t[CARS];
    for (long i = 0; i < CARS; i++)
        pthread_create(&t[i], NULL, car, (void *)(i + 1));
    for (int i = 0; i < CARS; i++)
        pthread_join(t[i], NULL);
    sem_destroy(&places);
    return 0;
}
```

Figure 2.7 shows the output. Three cars enter at once, and the other three enter only when a car leaves. The order of the cars can differ from one run to another, but the value of `inside` never goes above 3.

```
car 1 enters  (inside: 1)
car 4 enters  (inside: 2)
car 2 enters  (inside: 3)
car 2 leaves  (inside: 2)
car 5 enters  (inside: 3)
car 4 leaves  (inside: 2)
car 6 enters  (inside: 3)
car 1 leaves  (inside: 2)
car 3 enters  (inside: 3)
car 6 leaves  (inside: 2)
car 5 leaves  (inside: 1)
car 3 leaves  (inside: 0)
```

Figure 2.7: Output of `parking`: six cars, three places, never more than 3 inside

For the second question, a copy of the program (`parking1.c`) was made with `#define PLACES 1`. Its output is shown in Figure 2.8: the cars enter and leave strictly one by one.

```
#define PLACES 1
car 1 enters  (inside: 1)
car 1 leaves  (inside: 0)
car 2 enters  (inside: 1)
car 2 leaves  (inside: 0)
car 3 enters  (inside: 1)
car 3 leaves  (inside: 0)
car 4 enters  (inside: 1)
car 4 leaves  (inside: 0)
car 5 enters  (inside: 1)
car 5 leaves  (inside: 0)
car 6 enters  (inside: 1)
car 6 leaves  (inside: 0)
```

Figure 2.8: Output of `parking1` (`PLACES` = 1): only one car inside at a time

2.4.1 Observe Answers (Part 4)

1. **The largest value of `inside` and the line that guarantees it:** the largest value is 3 (Figure 2.7). It is guaranteed by the line `sem_wait(&places);` at the beginning of `car()`, together with the start value given by `sem_init(&places, 0, PLACES);`. The semaphore has only 3 units, so the fourth car blocks in `sem_wait` until a car leaves and calls `sem_post`.

2. **What does the semaphore become with `PLACES` set to 1?** It becomes a binary semaphore, which behaves like a mutex (a lock): only one car can be inside at a time, so the cars enter and leave one by one and `inside` is never larger than 1 (Figure 2.8).

3 CONCLUSIONS

In this laboratory work I saw a race condition with my own eyes: two threads that add to the same counter without protection lose work, because `counter++` is three steps for the processor (load, add, store) and the threads can be interleaved between them. The result was wrong and different at every run. Protecting the critical section with a mutex, or with a semaphore that starts at 1, gave the right answer every time, at the price of a longer run time.

Furthermore, the same semaphore was used in three ways, and only its start value changed its role: started at 1 it is a lock, started at 0 it is a signal that makes one thread wait for another, and started at 3 it is a counter that allows at most three cars in the parking lot. The experiments with a wrong start value (2 for the lock, 1 for the signal) showed that the protection disappears immediately, so the start value of a semaphore must match the number of threads that are allowed to pass.